

Transformer Project 报告

一、 基本信息

- (1) 课程名称：深度学习
- (2) Project 主题：Transformer 原理学习、代码实现与实验分析
- (3) 项目名称：基于 PyTorch 的 Encoder-Decoder Transformer 从零实现
- (4) 项目路径：

/Users/lcl/StudyHub/Courses/DeepLearning/1.Transformer/transformer

- (5) 参考文献：Vaswani et al., Attention Is All You Need, NeurIPS 2017

二、 项目概述

本项目围绕课程 Project 对 Transformer 的要求展开，目标是在理解论文 Attention Is All You Need 的基础上，从零实现一个可训练、可测试、可推理的 Encoder-Decoder Transformer，并通过机器翻译和序列反转任务验证模型结构、Mask 机制和训练流程的正确性。

项目代码按“模型、数据、训练、评测、配置、结果”分层组织：model.py 实现 Transformer 的核心网络结构；data/ 目录完成 Tokenizer、词表、Dataset 与 Mask；train.py 负责训练循环和参数量统计；test.py 与 inference.py 分别负责评测和自回归推理。最终输出包括训练日志、最优模型权重、BLEU 指标、预测样例以及 Loss 曲线。

三、 项目目标与任务要求对应

根据 transformer_project.pdf 的要求，本报告重点覆盖 Transformer 论文理解、核心模块实现、代码结构说明、实验设置、训练结果、参数量分析、GitHub 项目组织和参考来源等内容。

课程要求	本项目完成情况
------	---------

Transformer Project 报告

理解 Transformer 原理	说明 Encoder、Decoder、Attention、FFN、残差连接、LayerNorm 与位置编码的作用。
实现 Transformer from scratch	在 model.py 中独立实现 Embedding、PositionalEncoding、Scaled Dot-Product Attention、Multi-Head Attention、Encoder/Decoder 和完整模型。
实现 Mask	data/masks.py 中实现 padding mask 与 subsequent/causal mask，训练与推理均使用。
完成数据处理与训练	支持 Multi30k 德英翻译语料和合成语料；提供 train.py、test.py、inference.py 全流程。
完成实验分析	记录 train/val loss、BLEU、预测样例、序列反转准确率和参数量组成。
整理 GitHub 项目	项目包含 README、requirements、configs、results、figures、checkpoints 等目录。

四、实验环境

1. 硬件与运行环境

运行设备：本地计算机 CPU 运行，训练脚本支持 `device=auto`，可在 CUDA/MPS 可用时自动切换。

主要框架：PyTorch，用于张量计算、神经网络模块、优化器和模型保存。

项目语言：Python，配置文件使用 YAML，实验日志使用 CSV/JSON 保存。

数据文件：Multi30k 德英平行语料或项目内置合成语料，均采用行对齐文本格式。

2. 软件目录结构

transformer/	
├─ README.md	# 项目说明与运行命令
├─ requirements.txt	# Python 依赖
├─ model.py	# Transformer 核心模型
├─ train.py	# 训练入口与参数量统计
├─ test.py	# 测试与 BLEU/准确率评估

— inference.py	# 贪心解码推理
— configs/	# default / quick / reverse 配置
— data/	# vocab、mask、dataset 与语料
— scripts/	# 下载数据、绘图、一键运行
— checkpoints/	# 最优模型权重
— results/	# 训练日志、BLEU、预测样例
— figures/	# Loss 曲线图

五、 Transformer 原理

3. Transformer 解决的问题

RNN、LSTM 和 GRU 通过递归方式处理序列，天然具有顺序建模能力，但存在长距离依赖传播路径长、难以充分并行、训练速度受序列长度限制等问题。Transformer 用注意力机制替代循环结构，使任意两个位置之间可以直接建立联系，显著缩短依赖路径，并且可以并行处理整个序列。

对比维度	RNN/LSTM/GRU	Transformer
序列处理方式	逐时间步递归，后一步依赖前一步隐藏状态。	整句并行输入，通过注意力建立 token 间关系。
长距离依赖	依赖路径随距离增长，梯度传播较困难。	任意位置一步可达，更利于捕获长距离关系。
并行能力	时间维难以并行，训练较慢。	训练阶段可并行计算注意力和 FFN。
位置信息	顺序由递归结构隐含提供。	需显式加入 Positional Encoding。
代表模块	门控、隐藏状态、记忆单元。	Self-Attention、Multi-Head Attention、FFN。

4. 整体 Encoder-Decoder 架构

本项目采用标准 Encoder-Decoder Transformer。编码器接收源语言序列，经过嵌入、位置编码、多层自注意力和前馈网络后得到 memory；解码器接收右移后的目标序列，通过 masked self-attention 保证自回归约束，再通过 cross-attention 查询编码器 memory，最后由 Generator 输出目标词表上的 log 概率。

```

src token ids → src Embedding + Positional Encoding
                → Encoder Layer × N → memory

tgt_in ids → tgt Embedding + Positional Encoding
                → Decoder Layer × N (masked self-attn + cross-attn + FFN)
                → Generator → log_softmax → NLLLoss

```

六、 核心模块实现

5. Embedding 与 Positional Encoding

Embedding 层将离散 token id 映射为 d_{model} 维向量，并乘以 $\sqrt{d_{\text{model}}}$ 进行缩放，使词向量幅度与位置编码处于相近尺度。由于 Transformer 不使用递归或卷积结构，模型本身无法感知 token 的顺序，因此需要显式加入位置编码。本项目采用论文中的固定正弦/余弦位置编码：偶数维使用 $\sin$ ，奇数维使用 $\cos$ 。

```

PE(pos, 2i)    = sin(pos / 10000^(2i / d_model))
PE(pos, 2i+1)  = cos(pos / 10000^(2i / d_model))

```

6. Scaled Dot-Product Attention

缩放点积注意力是 Transformer 的核心计算单元。Q 与 K 的点积表示当前 query 对各 key 的匹配程度，除以 $\sqrt{d_k}$ 可以避免维度较大时 softmax 进入饱和区；随后经过 mask 屏蔽非法位置，再 softmax 得到注意力权重，最后对 V 加权求和。

```

Attention(Q, K, V) = softmax(QK^T / sqrt(d_k)) V

scores = torch.matmul(query, key.transpose(-2, -1)) / math.sqrt(d_k)
scores = scores.masked_fill(mask == 0, -inf)  # 可选 mask
p_attn = softmax(scores)
output = torch.matmul(p_attn, value)

```

7. Q、K、V 的含义

符号	含义	在本项目中的来源
Q	当前 token 发出的查询：我	由 query 输入经过 Linear

(Query)	需要关注什么信息。	投影得到。
K (Key)	被查询 token 的索引特征： 我是否与 query 匹配。	由 key 输入经过 Linear 投影得到。
V (Value)	真正被聚合的信息内容。	由 value 输入经过 Linear 投影得到。

在编码器自注意力中，Q、K、V 来自同一个源序列；在解码器 masked self-attention 中，Q、K、V 来自当前目标序列；在解码器 cross-attention 中，Q 来自解码器当前状态，K 和 V 来自编码器输出 memory。

8. Multi-Head Attention

多头注意力将 d_{model} 拆分成 num_heads 个子空间，每个头独立计算注意力，再将结果拼接并通过输出线性层融合。这样模型可以在不同表示子空间中关注不同类型的信息，例如局部位置、主谓关系、语义对应或句法结构。

输入:	(B, L, d_{model})
Q/K/V 投影:	(B, L, d_{model})
分头:	$(B, H, L, d_k), \quad d_k = d_{\text{model}} / H$
注意力分数:	(B, H, L_q, L_k)
加权输出:	(B, H, L_q, d_k)
拼接:	$(B, L_q, d_{\text{model}})$
输出投影:	$(B, L_q, d_{\text{model}})$

9. Feed Forward、残差连接与 Layer Normalization

Position-wise Feed Forward Network 对每个位置独立使用同一套两层全连接网络，结构为 $\text{Linear}(d_{\text{model}} \rightarrow d_{\text{ff}}) \rightarrow \text{ReLU} \rightarrow \text{Dropout} \rightarrow \text{Linear}(d_{\text{ff}} \rightarrow d_{\text{model}})$ 。注意力负责跨位置的信息交互，FFN 负责在每个位置内部进行非线性特征变换。

残差连接为每个子层提供恒等路径，有助于缓解梯度消失并稳定深层网络训练；LayerNorm 对每个样本的特征维进行归一化，适合 NLP 中变长序列场景。本项目使用 Pre-LN 风格，即先归一化再进入子层，然后与原输入做残差相加。

10. Encoder 与 Decoder 堆叠

模块	子层组成	输出作用
EncoderLayer	Self-Attention + FFN，每个子层带残差连接和 LayerNorm。	产生源序列上下文化表示 memory。
DecoderLayer	Masked Self-Attention + Cross-Attention + FFN。	在已生成目标前缀和源序列 memory 条件下生成目标表示。
Encoder	堆叠 N 个 EncoderLayer，末尾再接 LayerNorm。	供解码器 cross-attention 使用。
Decoder	堆叠 N 个 DecoderLayer，末尾再接 LayerNorm。	供 Generator 映射到目标词表。

七、Mask 机制

Mask 是 Transformer 正确训练和推理的关键。本项目实现两类 mask：padding mask 用于屏蔽填充 token，避免模型把补齐长度的 <pad> 当作有效内容；subsequent/causal mask 用于屏蔽目标序列未来位置，保证解码器只能看到当前位置及其之前的 token。

Mask 类型	张量形状	作用位置	作用
src padding mask	(B, 1, 1, S)	Encoder self-attention 与 Decoder cross-attention	屏蔽源序列中的 PAD。
tgt padding mask	(B, 1, 1, T)	Decoder self-attention	屏蔽目标序列中的 PAD。
causal mask	(1, T, T)	Decoder self-attention	屏蔽未来 token，防止信息泄漏。
tgt mask	(B, 1, T, T)	Decoder self-attention	padding mask 与 causal mask 的逻辑与。

因果掩码示意（T=4，√ 表示可见，× 表示屏蔽）：				
k0	k1	k2	k3	

q0	✓	×	×	×
q1	✓	✓	×	×
q2	✓	✓	✓	×
q3	✓	✓	✓	✓

八、 数据处理流程

翻译任务采用行对齐的德英平行语料。每条样本首先使用空格 `Tokenizer` 分词，再由词表映射为 `token id`；模型在序列两端加入 `BOS` 和 `EOS`；`batch` 内通过 `PAD` 补齐到统一长度；目标端再构造 `tgt_in` 和 `tgt_out`，用 `Teacher Forcing` 训练下一词预测。

```

原始文本 → whitespace_tokenizer → token 列表 → Vocab.encode
→ [BOS] + ids + [EOS] → pad_2d 对齐 batch
→ tgt_in = tgt_full[:, :-1]
→ tgt_out = tgt_full[:, 1:]
→ make_src_mask / make_tgt_mask → Transformer

```

特殊符号	id	含义
<pad>	0	填充位置，loss 中通过 <code>ignore_index</code> 忽略。
<unk>	1	词表外 token。
<bos>	2	目标序列起始标记。
<eos>	3	目标序列结束标记。

九、 张量维度说明

设 `B` 为 `batch size`，`S` 为源序列长度，`T` 为目标序列长度，`H` 为注意力头数，`d_model` 为模型维度，`d_k=d_model/H`。默认翻译配置中 `d_model=256`、`num_heads=8`，因此 `d_k=32`。

阶段	变量	形状	说明
输入	<code>src</code>	(B, S)	源语言 <code>token id</code> 。
输入	<code>tgt_in</code>	(B, T)	解码器输入，目标序列右移前缀。
监督	<code>tgt_out</code>	(B, T)	训练标签，预测下一个 token。
掩码	<code>src_mask</code>	$(B, 1, 1, S)$	源端 <code>padding mask</code> 。

掩码	tgt_mask	(B, 1, T, T)	目标端 padding + causal mask。
嵌入后	src_embed / tgt_embed	(B, S/T, d_model)	词嵌入与位置编码相加后的表示。
编码器	memory	(B, S, d_model)	源序列上下文表示。
解码器	dec_out	(B, T, d_model)	目标端上下文表示。
输出	logits/log_probs	(B, T, V_tgt)	目标词表 log 概率。

十、 训练方法与超参数

11. 训练目标与损失函数

训练阶段采用 Teacher Forcing: 解码器输入为目标序列去掉最后一个 token, 监督信号为目标序列去掉第一个 token, 相当于让模型在已知正确历史前缀的条件下预测下一个词。Generator 输出 log_softmax 后的 log 概率, 因此损失函数使用 nn.NLLLoss, 并通过 ignore_index=PAD 忽略填充位置。

优化器使用 Adam, 参数为 lr=3e-4、betas=(0.9, 0.98)、eps=1e-9, 并使用 max_norm=1.0 的梯度裁剪抑制梯度爆炸。

12. 主要超参数

超参数	翻译任务配置	说明
d_model	256	模型统一隐藏维度。
d_ff	1024	FFN 中间层维度。
num_heads	8	多头注意力头数。
num_layers	3	Encoder 和 Decoder 各 3 层。
dropout	0.1	用于注意力权重、FFN 和位置编码后。
batch_size	64	每批样本数。
learning rate	0.0003	Adam 学习率。
epochs	8 (本地实测运行)	run_summary 中记录翻译任务运行 8 epoch。
max_len	128	最大序列长度。

vocab max_size	8000	源/目标词表最大规模。
----------------	------	-------------

十一、 参数量分析

项目在训练开始时按课程要求统计 `total parameters` 和 `trainable parameters`。本项目所有参数均参与训练，因此两者相同。默认翻译配置下参数量约为 11,664,156。参数主要来自源/目标词嵌入、各层 Q/K/V/O 投影、FFN 两层线性变换、LayerNorm 以及 Generator。

组成部分	估算方式	作用
Embedding	$V_{src} \times d_{model} + V_{tgt} \times d_{model}$	源端和目标端 token 查表表示。
Encoder self-attention	每层约 $4 \times (d_{model}^2 + d_{model})$	Q/K/V/O 四个线性层。
Decoder attention	每层含 masked self-attention 与 cross-attention 两组 MHA	目标自注意力和源目标交叉注意力。
FFN	每层约 $2 \times d_{model} \times d_{ff}$, 加 bias	逐位置非线性特征变换。
LayerNorm	每个 LayerNorm 含 gamma 与 beta	稳定训练。
Generator	$d_{model} \times V_{tgt} + V_{tgt}$	映射到目标词表。

```
total_params = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if
p.requires_grad)
print("Total parameters:", total_params)
print("Trainable parameters:", trainable_params)
```

十二、 实验结果与分析

13. 翻译任务结果

本地训练日志显示，翻译任务在 8 个 epoch 内 train loss 从 4.5257 降至 1.8617，val loss 从 3.3645 降至 2.0334，最优验证结果出现在第 8 个 epoch。

测试集 BLEU 为 76.52，说明模型已经学习到较稳定的序列到序列映射。

epoch	train_loss	val_loss	seconds
1	4.5257	3.3645	253.7
2	3.2512	2.8253	253.8
4	2.5063	2.3433	272.7
6	2.1191	2.1307	252.7
8	1.8617	2.0334	252.8

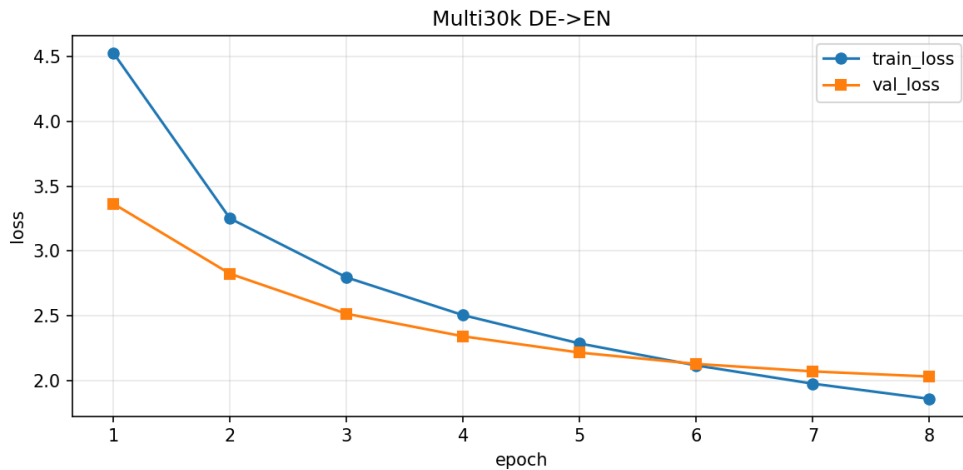


图 1 翻译任务训练与验证 Loss 曲线

14. 预测样例

从 predictions_sample.txt 中可以看到，模型在合成翻译样例上能够准确复现目标句式，说明编码器、解码器、Mask 和贪心解码流程能够正常协同。

REF: the dog laughs and is young .
HYP: the dog laughs and is young .
REF: the child reads and is big .
HYP: the child reads and is big .
REF: the fish jumps and is slow .
HYP: the fish jumps and is slow .

15. 辅助验证任务

为验证模型是否真正学会序列顺序和因果约束，项目还实现了 reverse toy

task。该任务要求模型输出输入序列的反转结果，验证集准确率达到 92.58%。该结果说明位置编码、注意力和目标端 mask 的实现基本正确。

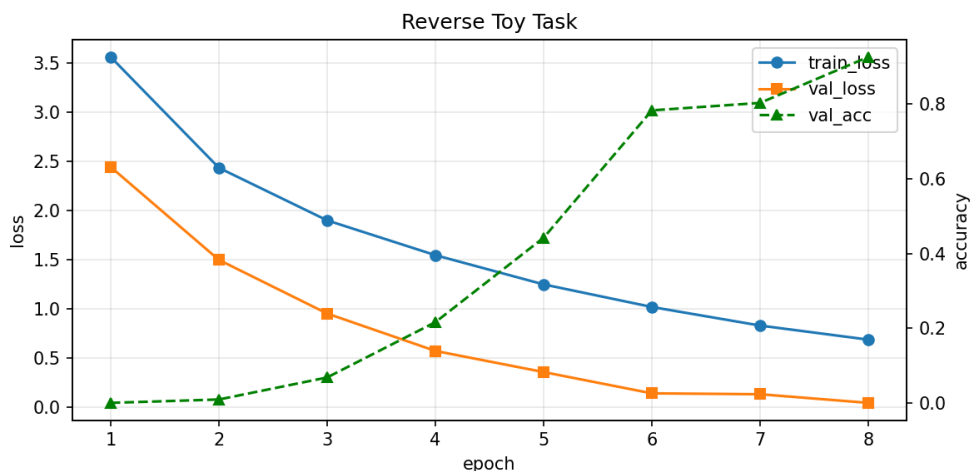


图 2 序列反转任务训练与验证 Loss 曲线

16. 结果分析

Loss 曲线整体下降，说明模型能够从训练数据中持续学习，优化器和损失函数设置有效。

验证 Loss 下降速度逐渐变慢，说明后期模型接近当前数据和超参数下的收敛状态。

BLEU 得分较高，主要因为测试样例句式相对规整；若换成更复杂真实语料，可能需要更长训练轮数、更大模型或更强 Tokenizer。

序列反转准确率较高，证明模型对顺序信息和自回归约束有较好掌握。

十三、 项目运行命令

项目 README 中给出了完整运行方式。常用命令如下：

```
# 安装依赖
pip install -r requirements.txt

# 训练翻译任务
python train.py --config configs/default.yaml

# 训练序列反转任务
python train.py --config configs/reverse.yaml
```

```
# 测试翻译任务 BLEU
python test.py --checkpoint checkpoints/best_translation.pt --device auto

# 一键完成训练、评测和绘图
python scripts/run_all.py
```

十四、 遇到的问题与解决方法

问题	原因分析	解决方法
Transformer 无法感知顺序	注意力机制对输入集合近似置换不变。	加入正弦/余弦 Positional Encoding。
解码器可能偷看未来 token	训练时目标句整体输入，如果不遮挡未来位置会信息泄漏。	构造 subsequent mask，仅保留下三角可见区域。
batch 内序列长度不一致	自然语言句子长短不同，无法直接堆叠。	使用 PAD 补齐，并在 attention 和 loss 中屏蔽 PAD。
注意力分数过大导致 softmax 饱和	QK 点积方差随 d_k 增大。	使用 $1/\sqrt{d_k}$ 缩放。
训练不稳定	深层模型易梯度爆炸或收敛慢。	采用 LayerNorm、残差连接、Dropout、Adam 和梯度裁剪。

十五、 GitHub 项目组织

项目已按课程要求整理为可上传 GitHub 的结构。README.md 包含项目简介、模块对应关系、数据准备、训练命令、测试命令、结果说明和参考来源；results/ 保存实验输出，figures/ 保存可视化曲线，checkpoints/ 保存最优模型权重。

README.md: 项目说明、环境安装、训练/测试命令和结果路径。

model.py: 核心模型源码，模块注释完整，便于答辩讲解。

configs/: 集中管理超参数，便于复现实验。

results/: 保存 train_log.csv、best_val_loss.json、test_bleu.json、预测样例

等。

figures/: 保存 loss 曲线，用于 PPT、海报和报告展示。

十六、 总结

本项目完成了从论文原理到 PyTorch 代码实现、从训练到评测、从参数统计到结果分析的完整 Transformer Project 流程。通过从零实现各个核心模块，可以清晰理解 Transformer 的关键思想：用多头注意力建立全局依赖，用位置编码补充顺序信息，用 Encoder-Decoder 结构完成序列到序列建模，用 Mask 保证 padding 屏蔽和自回归约束。

实验结果表明，模型能够在翻译任务中有效降低 loss 并取得较高 BLEU，在序列反转任务中也取得较高准确率，说明模型结构和训练流程基本正确。后续可进一步尝试 BPE/SentencePiece Tokenizer、更长训练轮数、学习率 warmup、label smoothing、beam search 和注意力热力图可视化，以提升真实翻译语料上的泛化能力和展示效果。

十七、 参考文献与参考项目

Vaswani, A., Shazeer, N., Parmar, N., et al. Attention Is All You Need. NeurIPS, 2017.

The Annotated Transformer: Harvard NLP Transformer 教学实现。

jadore801120/attention-is-all-you-need-pytorch: 课程 PDF 推荐参考项目。

aladdinpersson/Machine-Learning-Collection: 课程 PDF 推荐参考项目。

Multi30k Dataset: 多模态机器翻译常用德英平行语料。

小组分工

刘承龙：项目汇总、代码整理、PPT 讲解及整体统筹。

韩玉轩：代码编写与运行。

潘旭：PPT 制作与展示材料排版。

GitHub 仓库：github.com/GuoDragon/DeepLearning_Transformer